

Project Details

The Ultimate Goal:

We want to optimize the environment for Largemouth Bass Aquaculture. The project simulates the lifetime a swarm in many 2D near-realistic ponds.

The PSO help the fishes survive in the given pond, while the EA select the best pond that yields the the swarm with highest survival rate, best health condition and using cheapest policies.

General Implementation:

1. We input some basic aquaculture info:
 - *max_budget*: maximum budget, if the cost > max_budget, EA eliminate that pond without running the simulation.
 - *initial_fish_population*: the starting number of Largemouth Bass we have.
 - *aquaculture_days*: number of days from start to harvest.
 - *pond_generations*: how many time you want EA to select.
 - *run_simulations*: the number of parallel timelines that we run all generations.
2. For each simulation:
 1. We randomize the Fish Traits for each individual fish in the initial population.
 2. We create a fish swarm with Swarm Traits. This swarm contains all those fishes we just created.
 3. EA Loop:
 1. We randomize initial N ponds population (each has a unique Pond Genotype).
 2. We simulate the swarm's behaviors (finding biggest food source while avoid harmful factor) in each pond using PSO for the entire run time.
 3. We do EA on the results of each pond. We do the fitness on the survival rate and average Fish Stats of each fish in the remaining population. We only select the top best half ponds and eliminate the other half. We do Crossover and Mutation of the selected Pond Genotypes.
 4. We start again with the same initial swarm, but this time with new ponds population. Stop until there is only one ponds or reach the input ponds generation.
3. Then, we select the best "champion" among all simulation (timeline). This will be the final best pond.

Detailed Implementation:

Time: The defined time in the simulation:

- TimeStep: the smallest unit of time. 1 TimeStep equals to 1 realistic hour of aquaculture.
- Each realistic aquaculture day shall have 24 timestep.
- Runtime: the total run time of a swarm in a pond.

$$\text{Runtime} = \text{Total TimeStep} = \text{Aquaculture Days} * 24$$

Fixed Unit Cost: The cost for the products and policies:

- Food_Price: \$0.10 / pellet
- Probiotic_Price: \$0.50 / pellet
- Oxygen_Price: \$2.00 / hour (time-step)

Fixed Pond Traits: The fixed configurations applied to all ponds generations that is implemented in an 2D environment:

- Dynamic Objects: Food Pellet, Probiotic Pellet, Fecal Drop, Dead Fishes, Oxygen Bubbles and Pollutants
- Harmful areas: NH₃ Area, Disease Area and Parasite Area.
- Static and Dynamic Underwater Obstacles to mimic a realistic pond, fishes cannot move through obstacles.
- A slow natural Oxygen spamming rate.
- A slow natural NH₃ spamming rate.

Pond Genotypes: The controllable configurations of a pond that are implemented in a 2D environment. This genotype is defined by a specific set of integers and enum variables that will be explicitly mutated and selected by the Evolutionary Algorithm (EA).

1. Food Policy Genes

- **Food_Interval (Integer: 1 to 24):** The number of TimeSteps between food drops.
- **Food_Quantity (Integer: 1 to 10):** The exact number of Normal Pellets per drop.
- **Food_Location (Enum: 0, 1, 2):** Where the food physically drops.
 - 0 = Middle
 - 1 = Corner
 - 2 = Random
- **Total_Food_Cost (Engine Calculated):** = (Runtime / Food_Interval) * Food_Quantity * Food_Price

2. Probiotic Policy Genes

- **Probiotic_Interval (Integer: 1 to 24):** The number of TimeSteps between medicine drops.
- **Probiotic_Quantity (Integer: 1 to 10):** The exact number of Probiotic Pellets per drop.
- **Probiotic_Location (Enum: 0, 1, 2):** Where the medicine physically drops.
 - 0 = Middle
 - 1 = Corner
 - 2 = Random
- **Total_Probiotic_Cost (Engine Calculated):** = (Runtime / Probiotic_Interval) * Probiotic_Quantity * Probiotic_Price

3. Oxygen-Pumping Policy Genes

- **Oxygen_Interval (Integer: 1 to 24):** The number of TimeSteps between pumps.
- **Oxygen_Duration (Integer: 1 to 4):** How many TimeSteps the pump stays active.
- **Oxygen_Location (Enum: 0, 1, 2):** Where the pumps are located.
 - 0 = Middle
 - 1 = Corner
 - 2 = Random
- **Total_Oxygen_Cost (Engine Calculated):** = (Runtime / Oxygen_Interval) * Oxygen_Duration * Oxygen_Price

EA Fitness Function:

1. The Gatekeeper (Pre-Simulation Check)

- Before dropping the swarm, calculate:

$$\text{Cost} = (\text{Food_Quantity} * \text{Food_Price}) + (\text{Probiotic_Quantity} * \text{Probiotic_Price}) + (\text{Oxygen_Duration} * \text{Oxygen_Cost})$$

- If Cost > Max_Budget, instantly skip the simulation, mark it as failure.

2. The 3 Pillars of Fitness (Post-Simulation Check)

If the pond passes the Gatekeeper, simulate the swarm. Afterward, calculate the three pillars of success:

- **S - Survival Rate:** Number of remaining fishes over initial fishes (from 0.00 to 1.00)
- **C – Swarm’s Health:** The normalized average (from 0.0 to 1.0) of HP, Energy, Fullness, Immunity, and Velocity of all individuals in the swarm.
- **E - Budget Efficiency:** How much of the allowed budget was saved.

$$E = (\text{Max_Budget} - \text{Cost}) / (\text{Max_Budget})$$

3. The Final Fitness Equation

The EA uses a weighted sum to select the champion.

$$\text{Fitness} = (W_1 * S) + (W_2 * C) + (W_3 * E)$$

- **The Weighting Rule:** The weights enforce biological priority.
 - W₁ (Survival Weight) must be massive (0.50).
 - W₂ (Condition Weight) should be moderate (0.35).
 - W₃ (Efficiency Weight) must be low (0.15).

Objects Interaction: How configurations of a pond interact with themselves.

- Food has amount value. Each fish eating reduces this value by one.
- Food expires after some timestep (if their food value is not 0) and becomes pollutant.
- Pollutant inherits food value to become pollutant value.
- The probiotic is like food but expires much faster, thus, easier to become pollutant.
- Oxygen bubbles can move randomly but does not expire.
- NH₃ Area can move randomly but expires. Once expired, it becomes Pollutants that inherits its 50% values before transformation.
- Pollutants and Oxygen Bubbles do not exist in NH₃ Area.
- Some fishes inside the swarm may drop Fecal, we should stack all Fecal in a radius into a big Fecal with total Fecal Value.
- Fecal also has expires after some timestep and becomes pollutant, which inherit fecal value.
- Dead Fishes decays fast and become pollutant.
- After some timestep, the pollutant has chance becomes Disease Area or Parasite Area (could be both with low chance). The radius of area scale with pollutant values.
- Disease Area and Parasite Area also decay over time-step, and disappear eventually.

Fish Traits: The fixed values of a single fish that cannot change. Each stat is randomized in a defined ranges at the beginning of the PSO.

- MouthSize: Determines the maximum size of prey the particle can consume.
- BodySize: The primary defensive stat. If a particle's body size is smaller than another's mouth width, they are a valid target for cannibalism.

Fish Stats: The values of a single fish that can dynamically change based through interaction. Each stat is randomized in a defined ranges at the beginning of the PSO.

- Oxygen: The oxygen level that a fish has. It naturally decays by timestep and decays faster while fish inside NH₃ bubbles, regain by absorbing at Oxygen bubbles. A fish dies (removed from the swarm and become a dead fish) when Oxygen ≤ 0 .
- HP: Health Point of a fish, a typical value in survival setup. Fish dies if HP ≤ 0 .
- Energy: It costs fish energy to move. While Energy ≤ 0 , fish's HP reduces every timestep. Eating food unit regain energy with defined amount.
- Fullness: Indicates how full a fish's stomach is. This value decays by every timestep. Regain for each eaten food unit. Stop eating when full. When it is ≤ 0 , lose HP instead. If it eats another fish (cannibal), it regains Fullness = that fish's BodySize.
- FecalRate: When fullness > 0 , fish has a chance to drop fecal every 2 timestep. The chance increase with the fullness level.
- CannibalRate: Inversely scale with the fullness. Each fish has chance to eat nearby valid fish for every timestep.
- Immunity: Resistance to disease. Fish's immunity reduces each timestep if it contacts with Disease area. Fish got infections when Immunity ≤ 0 . While in infections, its HP reduces by each timestep. Immunity can be temporarily (10 timestep) boosted up by consuming probiotics pellet.
- Velocity: The swimming speed of the fish, and the following reduction multiply:

- Reduce 50% if HP $\leq$ 50%.
- Reduce 50% if Energy $\leq$ 50%.
- Reduce 50% if Fullness $\leq$ 50%.

Fish States: The state of a fish that triggers or being triggered in some special conditions. All has initial default value is False.

- **isBoosting:** True for 5 timestep since a fish just eats probiotic food. Overeating probiotic does not stack effect but refresh the duration.
- **isInfected:** True if a fish is infected. An infected fish becomes the new infected source with a small Disease Area.
- **hasParasite:** True if a fish has parasites. While in this state, a fish has a high chance to overwrite the the PSO Swarm's action scrubbing themselves against hard surface like obstacles to get rid of them. Importantly, this state weakens and hurts the fish:
 - Reduce Fullness efficiency by 50%
 - Drains extra 50% fullness per time-step
 - Drains extra 50% energy cost
 - Drains HP every 3 time-step (mimic slow death)
 - Reduce 25% velocity
- **isRunning:** True if a fish feel threatens by another fish. While in this state, a fish has a high chance to overwrite the the PSO Swarm's action with running action away from the dangerous nearby valid targets (the one with MouthSize larger than the running's BodySize). Increase 50% velocity but cost 50% more energy per time-step, buff ends if running out of energy.
- **isHunting:** True if a fish trigger cannibal mode. While in this state, a fish has a high chance to overwrite the the PSO Swarm's action with hunting action on nearby valid targets (the one with BodySize smaller than the hunting's BodySize). Increase 50% velocity but cost 50% more energy per time-step, buff ends if running out of energy.

How Cannibal Happens: If a cannibal's center coordinate enters the BodySize radius of a smaller fish, the physical collision occurs: the smaller fish's HP drops to 0 (removed from swarm), and the cannibal's Fullness increases. The eaten fish is removed from the swarm, do not spam a dead body in this case.

Swarm Traits: The parameter of the PSO that control the interaction of the swarm in the environment during the aquaculture time. Each param has a dynamic weight to make interaction more realistic.

- **w (Inertia Weight):** The baseline momentum of the fish, causing it to maintain its current heading based on its current speed.
- **Sensitive Distance (The Vision Radius):** This is the absolute maximum range of the fish's senses. The simulation only calculates cFood, cHazard (Disease areas), and cRun (Cannibals) if those entities are *inside* this distance. Anything outside of this radius is invisible to the fish.

- **Social Distance (The Cohesion Radius):** If another fish is inside this radius (but outside the Selfish distance), the cSocial vector activates, and the fish will adjust its trajectory to follow the group.
- **Selfish Distance (The Separation Radius):** If fish get too close to each other, they need a small repulsion vector to avoid clipping into one another.
- **cFood (Food Vector):** The pull toward commercial food. This vector dynamically scales with the Fullness stat. It is heavily prioritized when Fullness and Energy are low. When Fullness is maximum, this vector drops to 0, and the fish stops eating.
- **cProbiotic (Probiotic Vector):** The drive to seek out and consume probiotic pellets. This vector dynamically scales based on the fish's Immunity stat and its current isBoosting state. It is heavily prioritized when a fish's Immunity is dropping (such as when in contact with a Disease area). It should instantly drop to 0 if the fish's isBoosting state is True, preventing the fish from wasting valuable medicine. The swarm must act quickly on this vector because probiotics expire much faster into pollutants.
- **cOxygen (Oxygen Vector):** The critical survival drive to navigate toward and absorb moving Oxygen Bubbles. Therefore, the cOxygen vector scales inversely with the fish's Oxygen level. This vector has higher priority over other foraging vectors (cFood and cProbiotic) when the oxygen stat falls below a critical threshold. It must also calculate a dynamic interception path with the moving Oxygen Bubble.
- **cSelfish (Separation Vector):** The drive to maintain personal space and actively push away from peers. It acts as the mathematical opposite of cSocial. This vector only triggers if a neighboring fish crosses the Selfish Distance threshold. When triggered, it applies a repulsion vector pushing the fish directly away from the intruder.
- **cSocial (Schooling Vector):** This keeps the swarm moving as a cohesive unit to optimize general searching.
- **cNH3 (Toxicity Evasion Vector):** Extreme urgency because NH₃ quickly drains the Oxygen. A swarm should rarely risk swimming through NH₃, unless they are extremely hungry and there is high-value food drop in NH₃ bubbles.
- **cDisease (Bacterial Evasion Vector):** The swarm might risk grazing the edge of this zone if it has the isBoosting state from a probiotic but otherwise should route around it.
- **cParasite (Parasite Evasion Vector):** The drive to navigate away from Parasite Areas. A swarm that does not fear this vector will rapidly burn out its energy reserves and die.
- **cRelief (Scrubbing Vector):** The desperate urge to physically rub against surfaces to remove ectoparasites. If many individual hasParasite == True, this vector's weight violently increases. The fish needs to scrub immediately, while suffering from the hunger, being slow, and painful HP drains, creating a realistic struggling and indecisive interaction.
- **cRun (Cannibal Evasion Vector):** A strict survival repulsion vector. The PSO constantly checks the MouthSize and BodySize traits of neighboring fish. If a neighbor's

MouthSize is strictly larger than the subject's BodySize, a massive negative weight is applied, forcing the smaller fish to flee.

- **cHunt (Cannibal Vector):** The drive to consume smaller swarm members. Triggers when Fullness ≤ 0 and a valid target (MouthSize $>$ Target.BodySize) is within Sensitive Distance. The weight of this vector dynamically scales inversely with the distance to the prey.